



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Alumno:

Lara Hernandez Angel Husiel - 320060829

Arquitectura Cliente-Servidor

Grupo: 02 - Semestre: 2027-1

Tarea 2:

Permisos en linux, root y no root.

Profesor:

MTRO. CARLOS ALBERTO ROMAN ZAMITIZ

Fecha de Entrega:

3 de Septiembre del 2026

Índice

1 **Actividad 1:** 1

 1.1 Comando fdisk: 1

 1.2 Comando fdisk -l: 1

 1.3 Comando whoami: 1

 1.4 Ejecucion del comando fdisk -l: 1

 1.5 Comando ls -l /sbin/fdisk: 2

 1.6 Comando chmod para activar el setuid: 3

 1.7 Regreso de permisos originales: 4

2 **Actividad 2:** 4

Referencias 6



1. Actividad 1:

1.1. Comando fdisk:

El comando fdisk en linux, es un comando que se coloca directamente en la terminal, esto es para poder crear, eliminar, redimensionar, y administrar particiones de disco del propio sistema, no obstante tenemos que tener en cuenta que el comando fdisk en cuestion de usuarios con permisos, es un comando que solo puede ser ejecutado por el usuario root, esto es debido a que el comando fdisk tiene acceso a la tabla de particiones del disco duro, y si un usuario sin permisos de administrador lo ejecuta, este no tendra acceso a la tabla de particiones y por lo tanto no podra ejecutar el comando.

1.2. Comando fdisk -l:

Al agregar el parametro -l al mismo comando de fdisk, este nos mostrara un listado de todas las particiones que se encuentran en el disco duro, ademas de que nos mostrara informacion adicional de cada particion, como lo es el tamaño de la particion, el tipo de particion, el sistema de archivos que tiene cada particion, y el estado de cada particion.

1.3. Comando whoami:

El comando whoami en linux, es un comando que se coloca directamente en la terminal, esto es para poder mostrar el nombre del usuario que esta actualmente logeado en el sistema.

```
angel@angelMacOS: ~  
angel@angelMacOS:~$ whoami  
angel  
angel@angelMacOS:~$  
  
root@angelMacOS: /home/a  
root@angelMacOS:/home/angel# whoami  
root  
root@angelMacOS:/home/angel#
```

Figura 1: Ejecución del comando whoami.

1.4. Ejecucion del comando fdisk -l:

En el caso de este comando para el usuario no root, como se muestra en la imagen en todos los casos nos salen los permisos denegados, dejando en claro que este usuario no tiene el nivel acceso necesario.



```
angel@angelMacOS: ~  
angel@angelMacOS:~$ fdisk -l  
fdisk: no se puede abrir /dev/loop0: Permiso denegado  
fdisk: no se puede abrir /dev/loop1: Permiso denegado  
fdisk: no se puede abrir /dev/loop2: Permiso denegado  
fdisk: no se puede abrir /dev/loop3: Permiso denegado  
fdisk: no se puede abrir /dev/loop4: Permiso denegado  
fdisk: no se puede abrir /dev/loop5: Permiso denegado  
fdisk: no se puede abrir /dev/loop6: Permiso denegado  
fdisk: no se puede abrir /dev/loop7: Permiso denegado  
fdisk: no se puede abrir /dev/vda: Permiso denegado  
angel@angelMacOS:~$
```

Figura 2: Ejecución del comando fdisk -l, para usuario no root.

En el caso del usuario root, se muestra como se deja ver las particiones del disco sin problema alguno:

```
root@angelMacOS: /home/angel  
root@angelMacOS:/home/angel# whoami  
root  
root@angelMacOS:/home/angel# fdisk -l  
Disco /dev/loop0: 68,95 MiB, 72294400 bytes, 141200 sectores  
Unidades: sectores de 1 * 512 = 512 bytes  
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes  
Tamaño de E/S (mínimo/óptimo): 512 bytes / 512 bytes  
  
Disco /dev/loop1: 4 KiB, 4096 bytes, 8 sectores  
Unidades: sectores de 1 * 512 = 512 bytes  
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes  
Tamaño de E/S (mínimo/óptimo): 512 bytes / 512 bytes  
  
Disco /dev/loop2: 236,07 MiB, 247537664 bytes, 483472 sectores  
Unidades: sectores de 1 * 512 = 512 bytes  
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes  
Tamaño de E/S (mínimo/óptimo): 512 bytes / 512 bytes  
  
Disco /dev/loop3: 502,96 MiB, 527392768 bytes, 1030064 sectores  
Unidades: sectores de 1 * 512 = 512 bytes  
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
```

Figura 3: Ejecución del comando fdisk -l, para usuario root.

1.5. Comando ls -l /sbin/fdisk:

Para el caso de este comando ls -l /sbin/fdisk, se ejecuta si o si en la terminal del usuario root, esto es para poder ver los permisos que tiene el comando fdisk, y como se puede ver en la imagen, el comando fdisk tiene permisos de lectura, escritura y ejecución para el usuario root, y permisos de lectura y ejecución para el grupo y para los demás usuarios.



```
root@angelMacOS: /home/angel
root@angelMacOS:/home/angel# ls -l /sbin/fdisk
-rwxr-xr-x 1 root root 133312 mar  6 10:00 /sbin/fdisk
root@angelMacOS:/home/angel#
```

Figura 4: Ejecución del comando `ls -l /sbin/fdisk`, para usuario root.

1.6. Comando `chmod` para activar el `setuid`:

Para el caso de este comando, se tiene que colocar en la terminal el siguiente comando `chmod 4511 /sbin/fdisk`, esto para poder activar el `setuid`, esto es para que el comando `fdisk` pueda ser ejecutado por cualquier usuario, y este tenga los permisos del usuario root, esto es debido a que el comando `fdisk` tiene acceso a la tabla de particiones del disco duro, y si un usuario sin permisos de administrador lo ejecuta, este no tendra acceso a la tabla de particiones y por lo tanto no podra ejecutar el comando.

```
root@angelMacOS:/home/angel# chmod 4511 /sbin/fdisk
root@angelMacOS:/home/angel# ls -l /sbin/fdisk
-r-s--x--x 1 root root 133312 mar  6 10:00 /sbin/fdisk
root@angelMacOS:/home/angel#
```

Figura 5: Ejecución del comando `chmod` para activar el `setuid`.

De igual forma volvemos a lanzar el comando del punto anterior, para corroborar que los permisos cambiaron.

Regresando al usuario no-root, ahora ejecutando el comando `fdisk -l`, podemos ver que ahora si nos deja ver las particiones del disco duro, esto es debido a que el comando `fdisk` tiene acceso a la tabla de particiones del disco duro.

```
angel@angelMacOS:~$ fdisk -l
fdisk: no se puede abrir /dev/loop0: Permiso denegado
fdisk: no se puede abrir /dev/loop1: Permiso denegado
fdisk: no se puede abrir /dev/loop2: Permiso denegado
fdisk: no se puede abrir /dev/loop3: Permiso denegado
fdisk: no se puede abrir /dev/loop4: Permiso denegado
fdisk: no se puede abrir /dev/loop5: Permiso denegado
fdisk: no se puede abrir /dev/loop6: Permiso denegado
fdisk: no se puede abrir /dev/loop7: Permiso denegado
fdisk: no se puede abrir /dev/vda: Permiso denegado
angel@angelMacOS:~$ fdisk -l
Disco /dev/loop0: 60,95 MiB, 7294400 bytes, 141200 sectores
Unidades: sectores de 1 * 512 = 512 bytes
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
Tamaño de E/S (mínimo/optimo): 512 bytes / 512 bytes

Disco /dev/loop1: 4 KiB, 4096 bytes, 8 sectores
Unidades: sectores de 1 * 512 = 512 bytes
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
Tamaño de E/S (mínimo/optimo): 512 bytes / 512 bytes

Disco /dev/loop2: 236,07 MiB, 247537664 bytes, 483472 sectores
Unidades: sectores de 1 * 512 = 512 bytes
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
Tamaño de E/S (mínimo/optimo): 512 bytes / 512 bytes

Disco /dev/loop3: 502,96 MiB, 527392768 bytes, 1030064 sectores
Unidades: sectores de 1 * 512 = 512 bytes
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
Tamaño de E/S (mínimo/optimo): 512 bytes / 512 bytes

Disco /dev/loop4: 41,6 MiB, 43022400 bytes, 85200 sectores
Unidades: sectores de 1 * 512 = 512 bytes
Tamaño de sector (lógico/físico): 512 bytes / 512 bytes
```

Figura 6: Ejecución del comando `fdisk -l`, para usuario no root, después de activar el `setuid`.



1.7. Regreso de permisos originales:

Por ultimo para poder regresar a los permisos originales que teniamos se tiene que ejecutar el comando `chmod 755 /sbin/fdisk`, esto es para poder regresar los permisos originales que teniamos, esto es debido a que el comando `fdisk` tiene acceso a la tabla de particiones del disco duro, y si un usuario sin permisos de administrador lo ejecuta, este no tendra acceso a la tabla de particiones y por lo tanto no podra ejecutar el comando.

```
root@angelMacOS:/home/angel# chmod 755 /sbin/fdisk
root@angelMacOS:/home/angel# ls -l /sbin/fdisk
-rwxr-xr-x 1 root root 133312 mar  6 10:00 /sbin/fdisk
root@angelMacOS:/home/angel#
```

Figura 7: Ejecución del comando `chmod` para regresar los permisos originales.

2. Actividad 2:

Enunciado: **Ahora haz que el programa sea de un usuario y trata de ejecutarlo como otro usuario. Modifica el bit “setuid” para que veas diferencia entre el usuario real y el usuario efectivo.**

El codigo para este caso es el del archivo `programa02_ids.c`, donde en este codigo se obtienen los ids efectivos correspondientes, el id del usuario real, el id del usuario efectivo, el id del grupo real y por ultimo el id del grupo efectivo, esto es para poder ver la diferencia entre el usuario real y el usuario efectivo.

Para poder ver primero quien tiene los derechos del archivo, se debe ejecutar el comando `ls -l programa02_ids.c`.

Como se ve en la terminal de la derecha, el que tiene el derecho del archivo es “angel”, donde este caso, ahora en la terminal de la izquierda que es la terminal siendo root, vamos a cambiar el dueño del archivo, esto por medio del comando `chown root:root programa2`, ahora si en la terminal de la derecha siendo la de angel, tratamos de correr el programa directamente nos mandara un error, esto debido a que el usuario angel no tiene los permisos para ejecutar el programa, esto es debido a que el dueño del archivo es root.

De hecho de igual forma comprobando con el comando `ls -l programa2`, vemos que ahora es root el dueño del programa.



```
root@angelMacOS: /home/angel/Documentos/Cliente-Ser...
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1# chown root:root programa2
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1# ls -l programa2
-rwxrwxr-x 1 root root 70496 sep  1 07:24 programa2
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1# chmod 700 programa2
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1# ls -l programa2
-rwx----- 1 root root 70496 sep  1 07:24 programa2
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1#

angel@angelMacOS: ~/Documentos/Cliente-Servidor/Lab...
a1$ ls -l programa2
-rwxrwxr-x 1 angel angel 70496 sep  1 07:24 programa2
angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1$ ./ programa2
bash: ./: Es un directorio
angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1$ ./programa2
bash: ./programa2: Permiso denegado
angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1$
angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1$
```

Figura 8: Ejecución del comando `ls -l programa02`, para ver los permisos del archivo.

Por ultimo ahora tenemos que activar el `setuid`, esto por medio del comando `chmod 4511 programa2`, esto es para que el programa pueda ser ejecutado por cualquier usuario.

```
rtual/tema1# chmod 4511 programa2
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1# ls -l programa2
-r-s--x--x 1 root root 70496 sep  1 07:24 programa2
root@angelMacOS:/home/angel/Documentos/Cliente-Servidor/LaboratorioVirtual/tema1#
```

Figura 9: Ejecución del comando `chmod` para activar el `setuid`.

Con esto como se muestra en la terminal, ahora el id del usuario real y el id del usuario efectivo son diferentes, esto es debido a que el programa esta siendo ejecutado por un usuario diferente al que tiene los permisos del archivo, esto es debido a que el programa esta siendo ejecutado por el usuario `angel`, pero el programa tiene los permisos del usuario `root`, así que durante la ejecución del programa, este se vuelve `root`, y por lo tanto el id del usuario efectivo es diferente al id del usuario real.

```
angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tem
angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tem
Vi a1$ ./programa2
Real user ID: 1000
Vi Effective user ID: 0
Real group ID: 1000
Effective group ID: 1000
Vi angel@angelMacOS:~/Documentos/Cliente-Servidor/LaboratorioVirtual/tem
a1$
```

Figura 10: Ejecución del programa para ver la diferencia entre el usuario real y el usuario efectivo.

Referencias

Shotts, W. (2019, 1 de marzo). *Chapter 9: Permissions - The Linux Command Line*. Consultado el 1 de septiembre de 2026, desde <https://linuxcommand.org/tlcl.php>